Name: Gayatri Dargu

Batch :B4

class: sy-b(68)

AIM :4. Numerical Computation with NumPy: Perform different computations using NumPy library.

THEORY QUESTIONS

1. Explain the role of NumPy in Python scientific computing ecosystem

NumPy plays a very important role in the Python scientific computing ecosystem because it provides support for large, multi-dimensional arrays and matrices along with a wide range of mathematical functions. It acts as the foundation for many other libraries like Pandas, Matplotlib, and Scikit-learn. As shown in your PDF, NumPy is widely used in data science, machine learning, and artificial intelligence because it offers efficient storage and fast numerical computations.

2. Why is NumPy faster than pure Python computations?

NumPy is faster than pure Python because it is implemented using optimized C libraries, which perform computations at a lower level. It also uses vectorized operations, meaning it can process entire arrays at once instead of using loops like Python lists. As shown in the PDF example, operations on NumPy arrays take microseconds compared to much slower execution with Python lists.

3. What types of mathematical operations can be performed using NumPy?

NumPy supports a wide variety of mathematical operations such as basic arithmetic operations (addition, subtraction, multiplication, division), statistical operations (mean, median, standard deviation, variance), and advanced mathematical functions like logarithms, exponentials, and trigonometric functions. It also supports matrix operations like matrix multiplication and dot products, making it very useful for scientific and engineering calculations.

4. Explain indexing and slicing in NumPy arrays

Indexing and slicing in NumPy arrays are used to access specific elements or subsets of data. Indexing allows us to access a single element using its position, while slicing allows us to extract a range of elements from the array. For example, we can access elements using indices like a[0] or select multiple elements using slicing like a[1:4]. These operations are fast and efficient, making data manipulation easy.

5. How does NumPy integrate with other libraries?

a. Pandas: NumPy provides the underlying data structure for Pandas DataFrames and Series. All numerical operations in Pandas are powered by NumPy arrays.

b. Matplotlib: NumPy arrays are used to generate numerical data for plotting graphs in Matplotlib.

c. Scikit-learn: Scikit-learn uses NumPy arrays for storing datasets and performing machine learning computations.

```
import numpy as np
```

```
# 1. Create array from list
a = np.array([1, 2, 3, 4])
print(a)
```

```
[1 2 3 4]
```

```
# 2. Arithmetic operations
print(a + 2)
print(a * 2)
```

```
[3 4 5 6]
[2 4 6 8]
```

```
# 3. Sum and mean
print(np.sum(a))
print(np.mean(a))
```

```
10
2.5
```

```
# 4. Reshape array
b = np.array([1,2,3,4,5,6])
print(b.reshape(2,3))
```

```
[[1 2 3]
 [4 5 6]]
```

```
# 5. Indexing
print(a[0])
```

```
1
```

```
# 6. Matrix operations
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

print(A + B)       # addition
```

```
print(A.dot(B))    # multiplication
```

```
[[ 6  8]
 [10 12]]
[[19 22]
 [43 50]]
```

```
# 7. Random numbers
print(np.random.rand(3))
```

```
[0.38836049 0.99669631 0.37469616]
```

```
# 8. Std and variance
print(np.std(a))
print(np.var(a))
```

```
1.118033988749895
1.25
```

```
# 9. Math functions
print(np.sqrt(a))
print(np.log(a))
```

```
[1.         1.41421356 1.73205081 2.        ]
[0.         0.69314718 1.09861229 1.38629436]
```

```
# 10. List vs Array
list1 = [1,2,3]
array1 = np.array([1,2,3])

print("List:", [i*2 for i in list1])
print("Array:", array1*2)
```

```
List: [2, 4, 6]
Array: [2 4 6]
```